

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И
МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена трудового Красного Знамени федеральное государственное
бюджетное
образовательное учреждение высшего образования
«Московский технический университет связи и информатики»**

Кафедра Математическая кибернетика и информационные технологии

**Лабораторная работа №5
Разработка веб-приложения на Spring Boot.
Основы Spring Security
по дисциплине
«Информационные технологии и программирование»**

Выполнил: студент гр. БВТ2403
Толибов Р.Ш.

Руководитель:

Москва, 2026 г.

Цель работы

Изучить основы обеспечения безопасности в Spring Boot-приложении: научиться настраивать аутентификацию и авторизацию пользователей, ограничивать доступ к ресурсам приложения, работать с ролями и механизмами защиты запросов, а также познакомиться с основными подходами к хранению состояния пользователя в защищенной системе.

Задачи

- Подключить Spring Security к существующему проекту системы уведомлений (лабораторная №4).
- Расширить модель пользователя: добавить поля password и role, создать перечисление UserRole.
- Реализовать CustomUserDetails (адаптер между сущностью User и UserDetails) и CustomUserDetailsService (загрузка пользователя из БД по email).
- Настроить шифрование паролей с помощью BCryptPasswordEncoder.
- Создать конфигурацию безопасности SecurityConfig: определить открытые и закрытые URL, настроить DaoAuthenticationProvider, отключить CSRF, включить HTTP Basic.
- Реализовать регистрацию пользователя (RegisterRequest, AuthService, AuthController) с сохранением зашифрованного пароля и роли ROLE_USER.
- Проверить работу HTTP Basic аутентификации и разграничение доступа по ролям (пользователь/администратор).
- Изучить теоретические основы: цепочка фильтров Spring Security, сессионная аутентификация, JWT (stateless).
- Выполнить самостоятельные задания по доработке функциональности.

Ход работы

0. Подготовка

Использовался проект spring-lab3-notifications, разработанный и расширенный в лабораторной работе №4. Приложение работало с PostgreSQL, содержало сущности User и Notification, репозитории, сервисы и REST-контроллеры. Перед началом работы создана резервная копия проекта и выделена отдельная ветка.

1. Подключение Spring Security

В файл pom.xml добавлена зависимость:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Для последующей работы с JWT также добавлены зависимости:

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.6</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
```

После запуска приложения в консоли появился автоматически сгенерированный пароль для пользователя user, и все URL стали закрытыми — это подтвердило, что Spring Security активирован.

2. Расширение модели пользователя

Существовавшая сущность User не имела полей для аутентификации. В неё добавлены поля:

```
@Column(nullable = false, unique = true)
private String email;
```

```

@Column(nullable = false)
private String password;

@Enumerated(EnumType.STRING)
@Column(nullable = false)
private UserRole role;

```

Создано перечисление UserRole:

```

package org.example.model.enums;
public enum UserRole {
    ROLE_USER,
    ROLE_ADMIN
}

```

Также созданы отдельные DTO для регистрации и логина:

RegisterRequest (содержит name, email, password с валидацией) и LoginRequest (содержит email, password).

3. Подготовка репозитория и загрузки пользователя

В UserRepository добавлен метод поиска по email:

```
Optional<User> findByEmail(String email);
```

Создан класс CustomUserDetails, реализующий интерфейс UserDetails. Он хранит ссылку на сущность User и адаптирует её под требования Spring Security:

```

@AllArgsConstructor
public class CustomUserDetails implements UserDetails {
    private final User user;
    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of(new SimpleGrantedAuthority(user.getRole().name()));
    }
    @Override
    public String getPassword() {
        return user.getPassword();
    }
    @Override
    public String getUsername() {
        return user.getEmail();
    }
    public User getUser() { return user; }
}

```

Создан CustomUserDetailsService, реализующий UserDetailsService:

```

@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsService {
    private final UserRepository userRepository;
}

```

```

    @Override
    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
        User user = userRepository.findByEmail(username)
            .orElseThrow(() -> new UsernameNotFoundException("Пользователь
не найден"));
        return new CustomUserDetails(user);
    }
}

```

4. Шифрование паролей и конфигурация безопасности

Создан класс SecurityConfig с необходимыми бинами:

```

@Configuration
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {
    private final CustomUserDetailsService customUserDetailsService;

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public DaoAuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(customUserDetailsService);
        provider.setPasswordEncoder(passwordEncoder());
        return provider;
    }

    @Bean
    public AuthenticationManager
authenticationManager(AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .requestMatchers("/users/**").hasAnyRole("USER", "ADMIN")
            );
    }
}

```

```

        .requestMatchers("/notifications/**").hasAnyRole("USER",
"ADMIN")
        .anyRequest().authenticated()
    )
    .sessionManagement(session -> session
        .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
    )
    .authenticationProvider(authenticationProvider())
    .formLogin(form -> form.disable())
    .httpBasic(httpBasic -> {});
    return http.build();
}
}

```

5. Регистрация пользователя NotificationService

Создан AuthService:

```

@Service
@RequiredArgsConstructor
public class AuthService {
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public void register(RegisterRequest request) {
        User user = new User();
        user.setName(request.getName());
        user.setEmail(request.getEmail());
        user.setPassword(passwordEncoder.encode(request.getPassword()));
        user.setRole(UserRole.ROLE_USER);
        user.setCreatedAt(LocalDateTime.now());
        userRepository.save(user);
    }
}

```

AuthController:

```

@RestController
@RequestMapping("/auth")
@RequiredArgsConstructor
public class AuthController {
    private final AuthService authService;

    @PostMapping("/register")
    public String register(@RequestBody @Valid RegisterRequest request) {
        authService.register(request);
        return "Пользователь зарегистрирован";
    }
}

```

Проверка: отправлен POST-запрос через Postman на <http://localhost:8080/auth/register> с JSON:

```
{
  "name": "Иван Иванов",
  "email": "ivan@example.com",
  "password": "qwerty123"
}
```

В базе данных появилась запись, пароль сохранён в виде BCrypt-хэша, роль – ROLE_USER.

6. Базовая аутентификация через HTTP Basic

Без авторизации запросы к /users/all и /notifications/all возвращали статус 401. В Postman на вкладке Authorization выбран тип Basic Auth, указаны Username: ivan@example.com и Password: qwerty123. После этого запросы успешно выполнились, вернув данные.

7. Авторизация по ролям

В конфигурации уже заданы правила: /admin/** доступен только для ROLE_ADMIN. Для демонстрации создан простой контроллер AdminController:

```
@RestController
@RequestMapping("/admin")
public class AdminController {
    @GetMapping("/ping")
    public String ping() {
        return "Только для администратора";
    }
}
```

8. Как работают фильтры Spring Security

В отчёт включено описание цепочки фильтров: DelegatingFilterProxy → FilterChainProxy → основные фильтры (BasicAuthenticationFilter, SecurityContextPersistenceFilter, ExceptionTranslationFilter, FilterSecurityInterceptor). Понимание фильтров необходимо для настройки JWT-фильтра.

9. Сессии в Spring Security

Описан stateful-подход: после успешной аутентификации объект Authentication сохраняется в SecurityContext, который помещается в HTTP-сессию. Клиент получает cookie JSESSIONID. При последующих запросах контекст восстанавливается. В текущей конфигурации сессии включены (SessionCreationPolicy.IF_REQUIRED).

10. JWT: stateless-подход

Создан JwtService для генерации токенов, JwtAuthController с эндпоинтом /auth/login, возвращающим JWT, и JwtAuthenticationFilter, который извлекает токен из заголовка Authorization: Bearer ... и устанавливает

аутентификацию в SecurityContext. Для stateless-режима необходимо изменить sessionCreationPolicy на STATELESS и добавить фильтр перед UsernamePasswordAuthenticationFilter. В рамках лабораторной работы реализован демонстрационный вариант (без полной проверки срока действия токена в фильтре – оставлено для самостоятельного задания).

11. Сравнение сессий и JWT

Составлена таблица сравнения (в тексте отчёта):

Сессии – stateful, хранятся на сервере, используют cookie, удобны для браузерных приложений.

JWT – stateless, клиент хранит токен, подходит для REST API и мобильных приложений, требует проверки подписи.

12. Проверка работы приложения

Все эндпоинты протестированы в Postman:

Метод	URL	Результат
POST	/auth/register	200 ОК, пользователь создан
GET	/users/all без авторизации	401 Unauthorized
GET	/users/all с Basic Auth	200 ОК, список пользователей
GET	/admin/ping с ролью USER	403 Forbidden
GET	/admin/ping с ролью ADMIN	200 ОК, сообщение
POST	/auth/login (для JWT)	возвращает JWT токен

Самостоятельные задания

1. Endpoint регистрации администратора – в AuthController добавлен метод /register/admin, который устанавливает роль ROLE_ADMIN. Доступ к нему ограничен только для существующих администраторов (можно через @PreAuthorize).
2. Проверка уникальности email – в AuthService перед сохранением проверяется userRepository.existsByEmail(...), и если email уже существует, выбрасывается исключение EmailAlreadyExistsException с ответом 409 Conflict.
3. Обработка ошибки при неверном логине/пароле – глобальный обработчик исключений перехватывает BadCredentialsException и возвращает JSON с сообщением «Неверный email или пароль».

4. Вынос маппинга `User → UserDto` – создан отдельный класс `UserMapper` со статическими методами `toDto(User)` и `toEntity(UserDto)`.
5. Метод `extractUsername()` в `JwtService` – реализован как `Jwts.parser().verifyWith(key).build().parseSignedClaims(token).getPayload().getSubject()`.
6. Проверка срока действия токена – добавлен метод `isTokenValid(String token, UserDetails userDetails)`, который сравнивает `username` и проверяет, не истек ли срок.
7. Полное переключение на JWT (stateless) – в `SecurityConfig` изменено: `sessionCreationPolicy(SessionCreationPolicy.STATELESS)`, добавлен `jwtAuthenticationFilter` через `addFilterBefore(...)`.
8. Защита метода удаления пользователя – над методом `deleteUser` в `UserService` добавлена аннотация `@PreAuthorize("hasRole('ADMIN')")`. При попытке вызова от пользователя с ролью `USER` выбрасывается `AccessDeniedException`.

Вывод

В ходе лабораторной работы были освоены основы Spring Security. Проект системы уведомлений был дополнен механизмами аутентификации и авторизации. Создана модель пользователя с паролем и ролью, реализована загрузка пользователя через `UserDetailsService`, настроен `DaoAuthenticationProvider` с шифрованием `BCrypt`. Определены правила доступа к URL: открытая регистрация, закрытые для аутентифицированных пользователей ресурсы `/users` и `/notifications`, административная зона `/admin`. Проверена работа HTTP Basic аутентификации и разграничение прав по ролям.

Рассмотрены теоретические основы: цепочка фильтров Spring Security, сессионное хранение состояния и stateless-подход на основе JWT. Выполнены самостоятельные задания, включая создание JWT-фильтра, проверку уникальности email, обработку ошибок и защиту методов через `@PreAuthorize`.

<https://github.com/TRuslan666/ITaP/tree/main/лаб5>